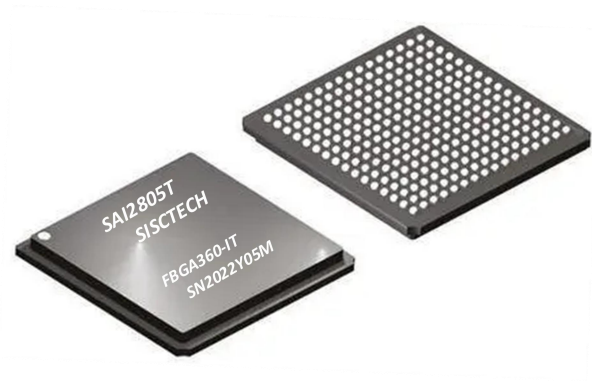




PiCube_Link

分布式对等互联协议

编写：edward yang

日期：2026 年 6 月 22 日

目录

第一章 术语定义	4
1.1 互联层级	4
1.2 互联链路	4
● 链路基础命令	4
● 数据下发命令	5
● 数据单目标回读命令	6
● 数据多目标回读命令	6
1.3 传输令牌	6
1.4 命令报文	7
1.5 数据报文	7
第二章 命令定义	8
2.1 命令的定义	8
2.2 命令的作用域定义	8
2.3 命令的功能定义	8
2.3.1 ROOT_ICFG/OCFG	8
2.3.2 ROOT_SIVT	8
2.3.3 ROOT_OVCT	8
2.3.4 ROOT_MIMT	8
2.3.5 ROOT_OMTX	8
2.3.6 ROOT_TASK	8
2.3.7 ROOT_NNEX	8
2.3.8 ROOT_ASET	8
2.3.9 ROOT_DNLD	8
2.3.10 ROOT_BOOT	9
2.3.11 HUBS_IVCT	9
2.3.12 HUBS_OVCT	9
2.3.13 HUBS_IMTX	9
2.3.14 HUBS_OMTX	9
第三章 构造分布式对等互联系统	10
3.1 建立 ROOT 与 HUBS 的连接	10
3.2 建立 HUBS 与 ROOT 的连接	10
3.3 配置 HUBS 的生态编号	11
3.4 配置 CORE 的生态编号	11
3.5 PTOP 下发权重	11
3.5 BATCH 下发权重	12
3.6 ROOT 广播下发矩阵数据	12
3.7 收集 CORE 矩阵数据到 HUB	13
3.8 收集 HUBS 矩阵数据到 ROOT	13
3.9 GAKV 广播下发矩阵数据	13

修订历史:

第一版 2026-06-22

第二版 2026-07-05 增加 GAKV 的自动 Gather-Degather 支持

第一章 术语定义

1.1 互联层级

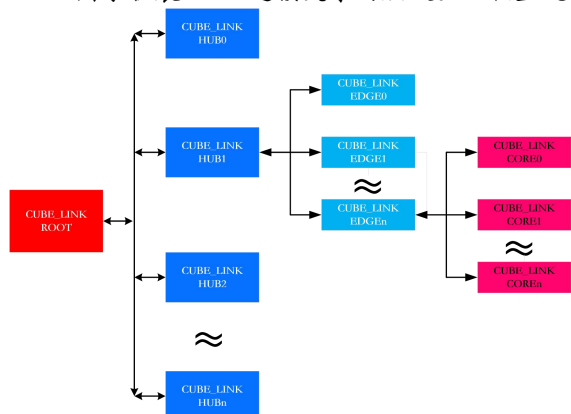
PiCUBE_LINK 是一种开放的分布式分层互联技术标准。用于将众多核心（CORE）通过标准互联手段汇聚到唯一的根（ROOT）控制节点。该标准的 4 层定义：

ROOT: 唯一的根控制节点，是访问整个互联系统的唯一控制端口。比较直观的例子是 PCIE 或 USB 接口；

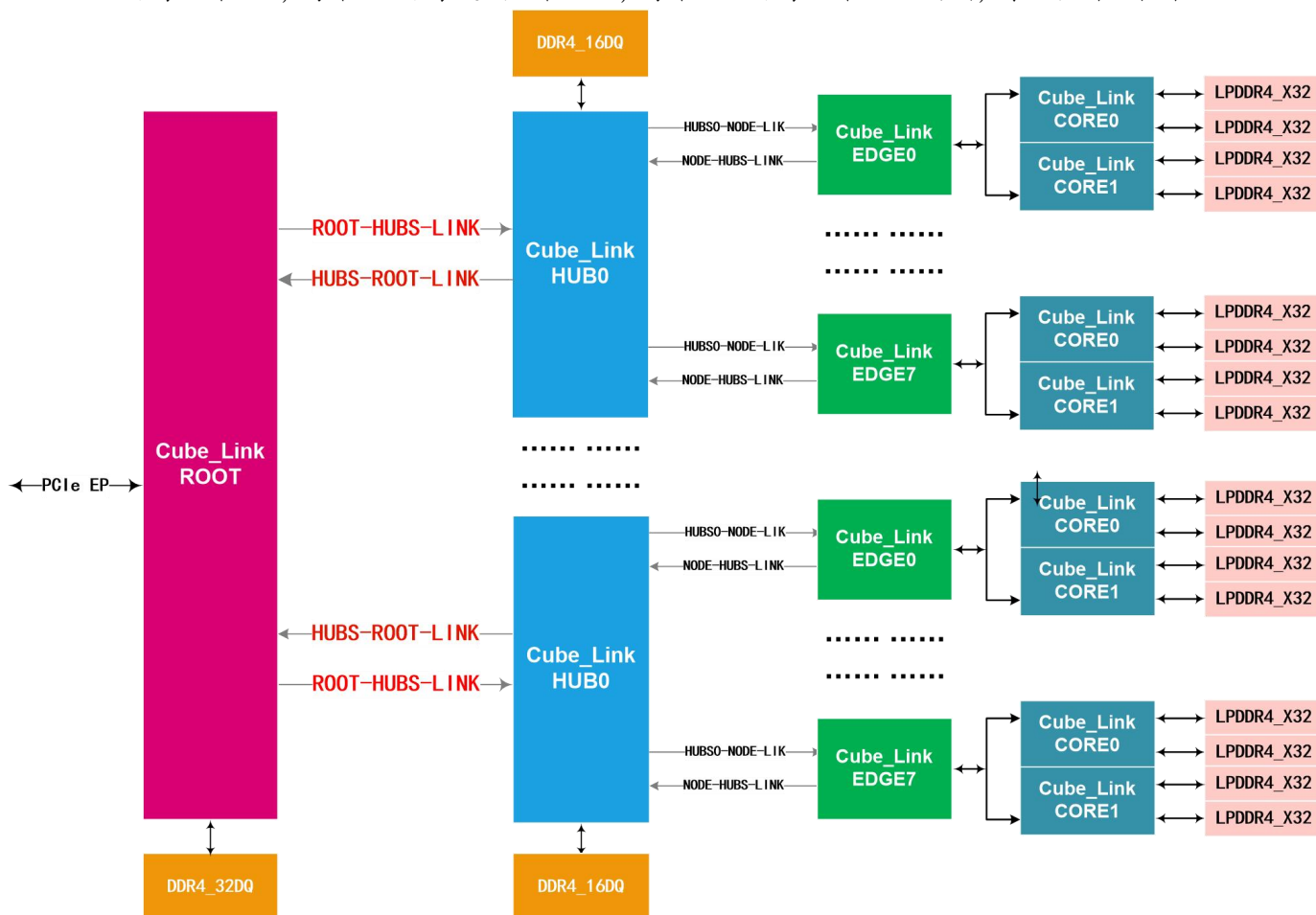
HUBS: 用于扩展 ROOT 连接关系的层级。比较直观的例子是 GPU 中的 GPC 实例；

EDGE: 用于扩展 HUBS 连接关系的层级。比较直观的例子是 GPC 中的 TPC 实例；

CORE: 用于扩展 EDGE 连接关系的层级。比较直观的例子是 TPC 中的 SM-TENSOR-CORE 实例；



以 ROOT 扩展 4 个 HUB，每个 HUB 扩展连接 8 个 EDGE，每个 EDGE 扩展 2 个 CORE 为例，系统框图见下图：



1.2 互联链路

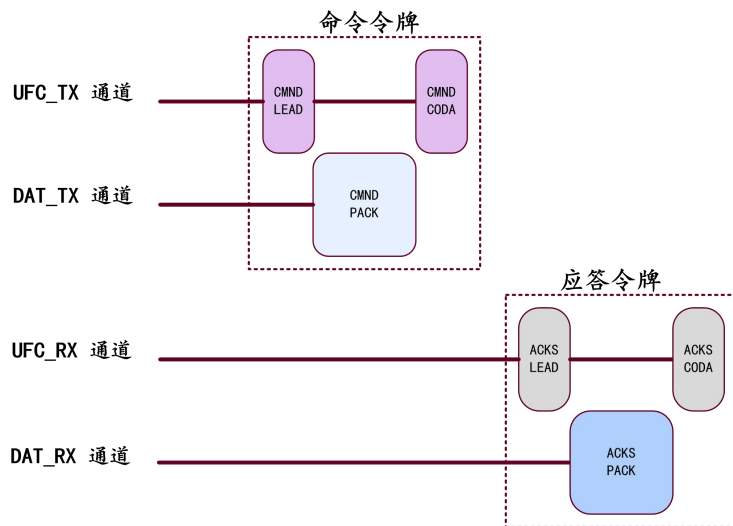
PiCUBE_LINK 的物理层电气连接上运行的是自定义的链路互联命令协议，分为 3 类链路互联命令，分别是：配置命令、数据下发命令、数据回读命令。3 种命令均有不同属性的令牌构成。

*处于上游的层级相对于下游的层级被称为主设备，处于下游的相连设备为从设备；

● 链路基础命令

由命令报文令牌和命令应答令牌构成：

ROOT 发送配置命令令牌给下游的 HUB，HUB 在处理该命令后，向 ROOT 发送 32 个 DWORD 的应答报文令牌



● 数据下发命令

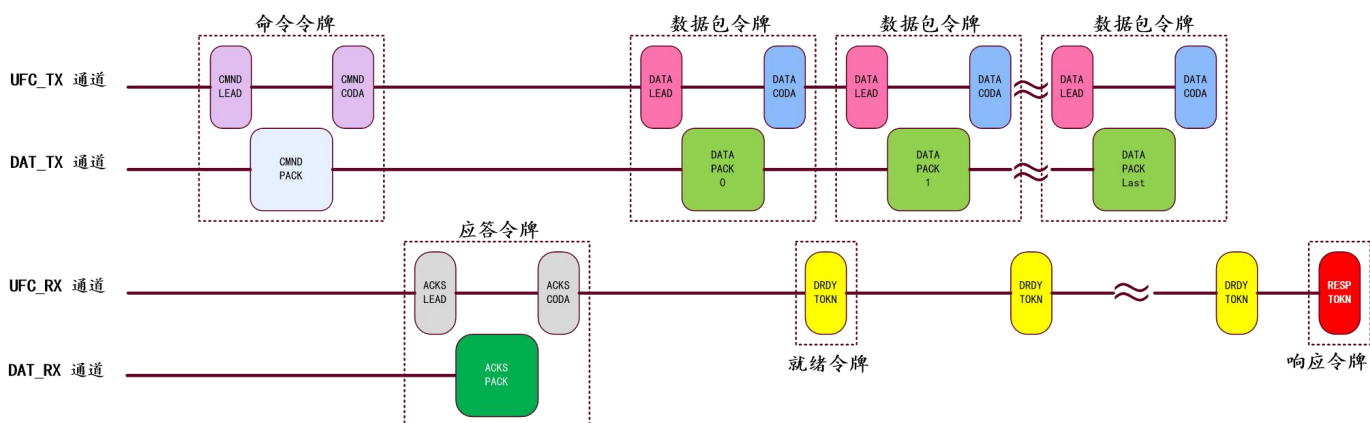
由命令报文令牌、命令应答令牌、数据报文令牌、就绪令牌、数据响应令牌构成：

ROOT 发送数据下发命令令牌给下游的 HUB，HUB 在处理该命令后，向 ROOT 发送应答令牌；

ROOT 在数据准备好后，发送数据包令牌；

HUBS 在收到每个数据包后，均需要向主机发送就绪令牌，用以实现 ROOT 和 HUBS 的传输背靠背反压；

HUBS 收到最后一个数据包，需向主机发送数据响应令牌，用以表明本次命令数据包被完全正确接收；



● 数据单目标回读命令

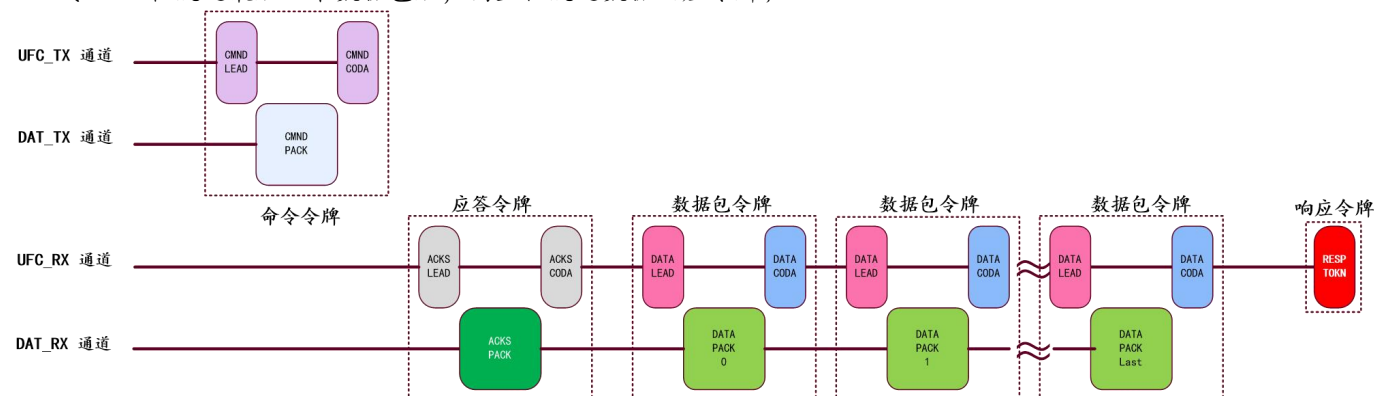
由命令报文令牌、命令应答令牌、数据报文令牌、数据响应令牌构成：

ROOT 发送数据回读令牌给下游的单一 HUB 或 CORE，

HUB 或 CORE 在处理该命令后，向 ROOT 发送应答令牌；

HUB 或 CORE 在数据准备好后，发送数据包令牌；

HUB 或 CORE 在发送最后一个数据包后，向主机发送数据响应令牌；



● 数据多目标回读命令

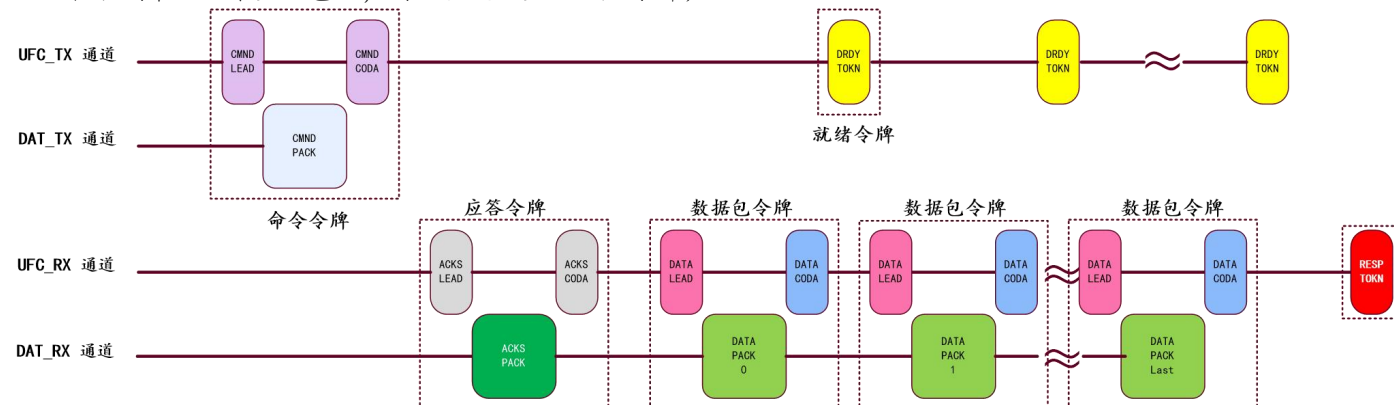
由命令报文令牌、命令应答令牌、数据报文令牌、就绪令牌、数据响应令牌构成：

ROOT 发送数据下发命令令牌给下游的 HUB，HUB 在处理该命令后，向 ROOT 发送应答令牌；

ROOT 在数据准备好后，发送数据包令牌；

HUBS 在收到每个数据包后，均需要向主机发送就绪令牌，用以实现 ROOT 和 HUBS 的传输反压；

HUBS 在收到最后一个数据包后，向主机发送数据响应令牌；



1.3 传输令牌

令牌按发送方向分为下行令牌和上行令牌。下行令牌由 ROOT 发送给 HUBS 或由 HUBS 发送给 EDGE；上行令牌由 HUBS 发送给 ROOT 或 EDGE 发送给 HUBS。每个令牌均由在 UFC 界面上传输的令牌头 (TKN_LEAD) 起始，跟随在 DAT 界面上传输的令牌报文数据 (TKN_PACK)，以及最后在 UFC 界面上传输的令牌尾 (TKN_CODA) 3 部分构成。

令牌类型	方向	令牌头	令牌报文	令牌尾	描述
命令令牌	下行	"CMNDLEAD"	08 个 DWORD	"CMNDCODA"	向下游发送命令代码
应答令牌	上行	"ACKSLEAD"	32 个 DWORD	"ACKSCODA"	向上游发送命令应答
响应令牌	上行	"RESPTKN"	NA	NA	向上游发送数据响应，作为数据包收发的结束标识
就绪令牌	上行	"DRDYTKN"	NA	NA	发送就绪状态，表明可以接受下一个数据包
数据包令牌	双向	"DATALEAD"	n 个 DWORD	"DATACODA"	上游（下游）向下游（上游）发送数据包

1.4 命令报文

报文双字	报文字段	缩写	定义
DWRD_00	WORD_00	CMND_LINK_HOST	定义链路命令的主机
	WORD_01	CMND_LINK_CODE	定理链路命令的类型
DWRD_01	WORD_00	HUBS_LINK_PORT	32 比特位域对应 32 个 HUBS 使能
	WORD_01	EDGE_LINK_PORT	32 比特位域对应 32 个 EDGE 使能
DWRD_02	WORD_00	HUBS_LINK_DEST	HUBS 的命令作用目标
	WORD_01	EDGE_LINK_DEST	EDGE 的命令作用目标
DWRD_03	WORD_00	CORE_LINK_DEST	CORE 的命令作用目标
	WORD_01	CMND_LINK_AUX1	命令辅助参数
DWRD_04	WORD_00	MAST_META_DDRS	内存地址（主）
	WORD_01	MAST_META_ROWS	数据行数（主）
DWRD_05	WORD_00	MAST_META_COLS	数据列数（主）
	WORD_01	MAST_CMND_AUX1	辅助参数（主）
DWRD_06	WORD_00	SLAV_META_DDRS	内存地址（从）
	WORD_01	SLAV_META_ROWS	数据行数（从）
DWRD_07	WORD_00	SLAV_META_COLS	数据列数（从）
	WORD_01	SLAV_CMND_AUX1	辅助参数（从）

1.5 数据报文

数据收发命令中的数据报文的个数，由命令参数 XXXX_META_ROWS 指定;单个的数据报文的长度为 n 个 DWORD，n 由命令参数 XXXX_META_COLS 决定，**当数据精度为 FP16 时，参数 XXXX_META_COLS 必须为 8 的倍数。**

第二章 命令定义

2.1 命令的定义

CUBE_LINK_HOST 和 CUBE_LINK_CODE 表达了当前传输命令，见下表：

序号	CUBE_LINK_HOST	CUBE_LINK_CODE	定义	用途
0	"ROOT"	"ICFG"	用于 ROOT 回读 HUBS, EDGE, CORE 配置	
1	"ROOT"	"OCFG"	用于 ROOT 配置 HUBS, EDGE, CORE 参数	
2	"ROOT"	"SIVT"	ROOT 从单目标 HUB 或 CORE 读 VECTOR 数据	
3	"ROOT"	"OVCT"	ROOT 向多目标 HUB 或 CORE 发 VECTOR 数据	
4	"ROOT"	"IMTX"	用于 ROOT 回读多目标 HUB 的 MATRIX 数据	
5	"ROOT"	"OMTX"	ROOT 下发 CORE 的单一 MATRIX 数据	
6	"ROOT"	"TASK"	用于 ROOT 发布 CORE 任务管理命令	
7	"ROOT"	"NNEX"	用于 ROOT 发布 CORE 算子调用命令	
8	"ROOT"	"ASET"	用于 ROOT 配置 HUBS, EDGE, CORE 的访问模式	
9	"ROOT"	"DOWN"	用于 ROOT 配置 HUBS, EDGE, CORE 的访问数据	
10	"ROOT"	"BOOT"	用于 ROOT 配置 HUBS, EDGE, CORE 的启动模式	
11	"HUBS"	"OVCT"	用于 HUBS 下发 CORE 的 VECT 数据	FU
12	"HUBS"	"IMTX"	用于 ROOT 回读 CORE 的 MTRX 数据	
13	"GAKV"	"OMTX"	用于 HUBS 下发 CORE 的 GAKV 数据	

2.2 命令的作用域定义

XXXX_LINK_DEST 表达了当前传输命令的作用域，见下表：

编号	HUBS_LINK_DEST	CORE_LINK_DEST	CORE_LINK_DEST
0	HUBS_DEST_SCUT	EDGE_DEST_SCUT	CORE_DEST_SCUT
1	HUBS_DEST_CRAM	EDGE_DEST_CRAM	CORE_DEST_CRAM
2	HUBS_DEST_CFIG	EDGE_DEST_CFIG	CORE_DEST_CFIG
3	HTOR_DEST_MGTH	EDGE_DEST_ISPM	CORE_DEST_DVPO
4	HTOE_DEST_MGTH	ETOH_DEST_MGTH	CORE_DEST_DVP1
5	HTOE_DEST_DDRS		CORE_DEST_DVPD
6			CORE_DEST_GML0
7	HUBS_DEST_SCFG		CORE_DEST_GML1
8	HUBS_DEST_CNTC		CORE_DEST_NNEX
9	HUBS_DEST_INDX		CORE_DEST_INDX

2.3 命令的功能定义

2.3.1 ROOT_ICFG/OCFG

- 仅有 ROOT 可以进行下游设备的参数配置。基础配置项包括：
- 配置或回读各个 EDGE_LINK 初始化状态（HUBS_DEST_MGTH），以建立 HUBS 与 EDGE 间的链路连接
 - 配置或回读各个 HUBS 的 ECOS 唯一标识（HUBS_DEST_ECOS）
 - 配置或回读各个 EDGE 的 ISPM 连接链路（EDGE_DEST_ISPM）
 - 配置或回读各个 CORE 的 ECOS 唯一标识（EDGE_DEST_ECOS）

2.3.2 ROOT_SIVT

由 ROOT 发起读取单目标的 HUBS, CORE 的 VECTOR 数据，**不支持多目标同时读取**

2.3.3 ROOT_OVCT

由 ROOT 向**单目标或多目标**的 HUBS, CORE 下发 VECTOR 数据

2.3.4 ROOT_MIMT

由 ROOT 从**单目标或多目标**的 HUBS, CORE 读取 MATRIX 数据，并根据指定参数实现数据的**自动交织**

2.3.5 ROOT_OMTX

由 ROOT 向**单目标或多目标**的 HUBS, CORE 发送块交织的 MATRIX 数据，交织维度是 128 列

2.3.6 ROOT_TASK

由 ROOT 向**单目标或多目标**的 CORE 发送任务管理命令

2.3.7 ROOT_NNEX

由 ROOT 向**单目标或多目标**的 CORE 发送算子调度命令

2.3.8 ROOT_ASET

由 ROOT 向**单目标或多目标**的 HUB 或 EDGE 发送访问模型配置命令

2.3.9 ROOT_DNLD

由 ROOT 向**单目标或多目标**的 HUB 或 EDGE 发送访问模型下的下发数据

2.3.10 ROOT_BOOT

由 ROOT 向**单目标或多目标**的 HUB 或 EDGE 发送访问模型下的启动请求

2.3.11 HUBS_OVCT

由 HUB 向连接的**单目标** CORE 发送 VECTOR 数据

2.3.12 HUBS_IMTX

由 HUB 从连接的**单目标或多目标**的 CORE 回读 MATRIX 数据，并根据指定参数实现数据的**自动交织**

2.3.13 GAKV_OMTX

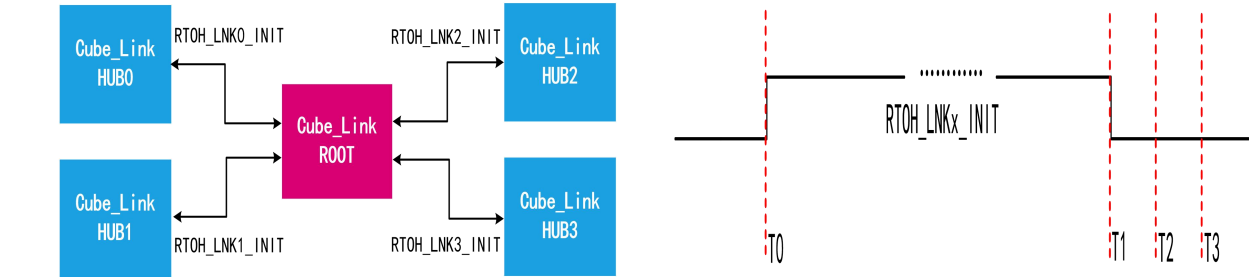
由 HUB 向连接的**单目标或多目标**的 CORE 发送 GAKV 数据

第三章 构造分布式对等互联系统

基于前文所述的互联结构，链路协议，CUBE_LINK 能将众多 CORE 组成成为一个由 ROOT 端口进行自由访问的分布式对等系统。下面以实现支持 TF 架构为目的的分布式对等系统，说明关键实现细节。

3.1 建立 ROOT 与 HUBS 的连接

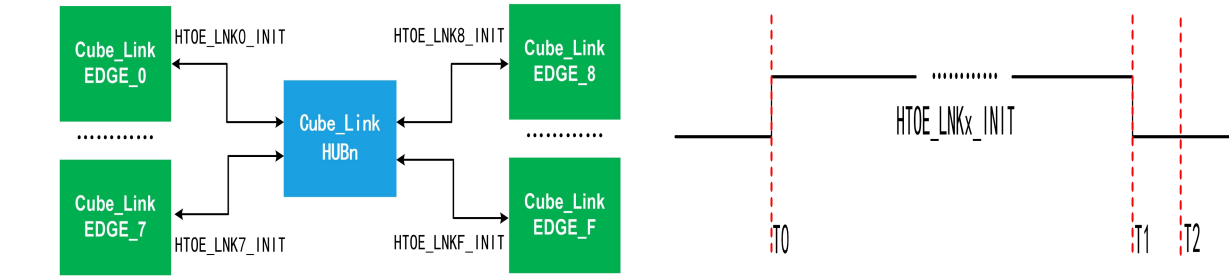
- ROOT 与 HUBS 之间通过 ROTH_LNKX_INIT 差分信号，来启动双方之间的 AURORA 互联的初始化。
- T0 时间 ROOT 向 HUBS 发出 ROTH_LNKX_INIT 的有效信号，在 ROOT 和 HUBS 的 AURORA 初始化电路探测到该信号的上升沿后，启动 AURORA 的复位控制流程；
- T1 时间 ROOT 向 HUBS 发出 ROTH_LNKX_INIT 的无效信号，在 ROOT 和 HUBS 的 AURORA 初始化电路探测到该信号的下降沿后，启动 AURORA 的复位撤销流程；
- T2 时间主机查看 ROOT 的各个 AURORA 链路状态，确认 ROOT 建立有效连接；
- T3 时间主机发送 ROOT_ICFG 命令，回读 HUBS 的 AURORA 链路状态，确认 HUBS 建立有效连接；



报文字段	缩写	CMD_T3
WORD_00	CMND_LINK_HOST	“ROOT_ICFG”
WORD_01	CMND_LINK_CODE	
WORD_02	HUBS_LINK_PORT	32’ H0000000F
WORD_03	EDGE_LINK_PORT	32’ H00000000
WORD_04	HUBS_LINK_DEST	HTOR_DEST_MGTH
WORD_05	EDGE_LINK_DEST	NA
WORD_06	CORE_LINK_DEST	NA
WORD_07	CMND_LINK_AUX1	NA
WORD_08~15	MAST_META_DDRS	NA

3.2 建立 HUBS 与 ROOT 的连接

- HUBS 与 EDGE 之间通过 HOTE_LNKX_INIT 差分信号，来启动双方之间的 AURORA 互联的初始化。
- T0 时间 ROOT 向 HUBS 发送 ROOT_OCFG 命令,HUBS 向 EDGE 发出 HOTE_LNKX_INIT 的有效信号，在 HUBS 和 EDGE 的 AURORA 初始化电路探测到该信号的上升沿后，启动 AURORA 的复位控制流程；
- T1 时间 ROOT 向 HUBS 发送 ROOT_OCFG 命令,HUBS 向 EDGE 发出 HOTE_LNKX_INIT 的无效信号，在 HUBS 和 EDGE 的 AURORA 初始化电路探测到该信号的下降沿后，启动 AURORA 的复位撤销流程；
- T2 时间 ROOT 向 HUBS 发送 ROOT_ICFG 命令,查看 HUBS 的各个 AURORA 链路状态，确认 HUBS 建立有效连接；
- T3 时间 ROOT 向 HUBS 发送 ROOT_ICFG 命令,查看 EDGE 的各个 AURORA 链路状态，确认 EDGE 建立有效连接；



报文字段	缩写	CMD_T0	CMD_T1	CMD_T2	CMD_T3
WORD_00	CMND_LINK_HOST	“ROOT_OCFG”	“ROOT_OCFG”	“ROOT_ICFG”	“ROOT_ICFG”
WORD_01	CMND_LINK_CODE				
WORD_02	HUBS_LINK_PORT	32’ H0000000F	32’ H0000000F	32’ H0000000F	32’ H0000000F
WORD_03	EDGE_LINK_PORT	32’ H000000FF	32’ H000000FF	32’ H00000000	32’ H000000FF
WORD_04	HUBS_LINK_DEST	HTOE_DEST_MGTH	HTOE_DEST_MGTH	HTOE_DEST_MGTH	HUBS_DEST_SCUT
WORD_05	EDGE_LINK_DEST	NA	NA	NA	ETOH_DEST_MGTH
WORD_06	CORE_LINK_DEST	NA	NA	NA	NA
WORD_07	CMND_LINK_AUX1	32’ H0000000X	32’ H0000000Y	NA	NA
WORD_08~15	MAST_META_DDRS	NA	NA	NA	NA

3.3 配置 HUBS 的生态编号

报文字段	缩写	CMD_T0
WORD_00	CMND_LINK_HOST	“ROOT_OCFG”
WORD_01	CMND_LINK_CODE	
WORD_02	HUBS_LINK_PORT	32’ H00000001
WORD_03	EDGE_LINK_PORT	32’ H00000000
WORD_04	HUBS_LINK_DEST	HUBS_ECOS_INDX
WORD_05	EDGE_LINK_DEST	NA
WORD_06	CORE_LINK_DEST	NA
WORD_07	CMND_LINK_AUX1	32’ HABCDEFGH
WORD_08~15	MAST_META_DDRS	NA

3.4 配置 CORE 的生态编号

报文字段	缩写	CMD_T0
WORD_00	CMND_LINK_HOST	“ROOT_OCFG”
WORD_01	CMND_LINK_CODE	
WORD_02	HUBS_LINK_PORT	32’ H00000001
WORD_03	EDGE_LINK_PORT	32’ H00000001
WORD_04	HUBS_LINK_DEST	HUBS_DEST_SCUT
WORD_05	EDGE_LINK_DEST	CORE_ECOS_INDX
WORD_06	CORE_LINK_DEST	NA
WORD_07	CMND_LINK_AUX1	32’ HABCDEFGH
WORD_08~15	MAST_META_DDRS	NA

3.5 PTOP 下发权重

报文字段	缩写	CMD
WORD_00	CMND_LINK_HOST	“ROOT_OVCT”
WORD_01	CMND_LINK_CODE	
WORD_02	HUBS_LINK_PORT	32’ H00000001
WORD_03	EDGE_LINK_PORT	32’ H00000001
WORD_04	HUBS_LINK_DEST	HUBS_DEST_SCUT
WORD_05	EDGE_LINK_DEST	EDGE_DEST_SCUT
WORD_06	CORE_LINK_DEST	CORE_DEST_DVPX
WORD_07	CMND_LINK_AUX1	NA
WORD_08	MAST_META_DDRS	ROOT_WGHT_DDRS
WORD_09	MAST_META_ROWS	ROOT_WGHT_ROWS
WORD_10	MAST_META_COLS	ROOT_WGHT_COLS
WORD_11	MAST_CMND_AUX1	NA
WORD_12	SLAV_META_DDRS	CORE_WGHT_DDRS

WORD_13	SLAV_META_ROWS	CORE_WGHT_ROWS
WORD_14	SLAV_META_COLS	CORE_WGHT_COLS
WORD_15	SLAV_CMND_AUX I	NA

3.5 BATCH 下发权重

将权重文件分别装载到各个 HUBS 的内存，然后用 HUBS_OVCT 指令启动 HUBS 对相连的单 CORE 进行权重下载。

报文 字段	缩写	CMD_1	CMD_2	CMD_3	CMD_4
WORD_00	CMND_LINK_HOST	“ROOT_OVCT”	“ROOT_OVCT”	“ROOT_OVCT”	“HUBS_OVCT”
WORD_01	CMND_LINK_CODE				
WORD_02	HUBS_LINK_PORT	32’ H00000001		32’ H00000008	32’ H0000000F
WORD_03	EDGE_LINK_PORT	32’ H00000000	32’ H00000000	32’ H00000000	32’ H00000001
WORD_04	HUBS_LINK_DEST	HUBS_DEST_DDRS	HUBS_DEST_DDRS	HUBS_DEST_DDRS	HUBS_DEST_DDRS
WORD_05	EDGE_LINK_DEST	NA	NA	NA	EDGE_DEST_SCUT
WORD_06	CORE_LINK_DEST	NA	NA	NA	CORE_DEST_DVPX
WORD_07	CMND_LINK_AUX I	NA	NA	NA	NA
WORD_08	MAST_META_DDRS	ROOT_WGHT_DDRS	ROOT_WGHT_DDRS	ROOT_WGHT_DDRS	HUBS_WGHT_DDRS
WORD_09	MAST_META_ROWS	ROOT_WGHT_ROWS	ROOT_WGHT_ROWS	ROOT_WGHT_ROWS	HUBS_WGHT_ROWS
WORD_10	MAST_META_COLS	ROOT_WGHT_COLS	ROOT_WGHT_COLS	ROOT_WGHT_COLS	HUBS_WGHT_COLS
WORD_11	MAST_CMND_AUX I	NA	NA	NA	NA
WORD_12	SLAV_META_DDRS	HUBS_WGHT_DDRS	HUBS_WGHT_DDRS	HUBS_WGHT_DDRS	CORE_WGHT_DDRS
WORD_13	SLAV_META_ROWS	HUBS_WGHT_ROWS	HUBS_WGHT_ROWS	HUBS_WGHT_ROWS	CORE_WGHT_ROWS
WORD_14	SLAV_META_COLS	HUBS_WGHT_COLS	HUBS_WGHT_COLS	HUBS_WGHT_COLS	CORE_WGHT_COLS
WORD_15	SLAV_CMND_AUX I	NA	NA	NA	NA

3.6 ROOT 广播下发矩阵数据

自动进行 128 列的块数据交织

报文 字段	缩写	CMD
WORD_00	CMND_LINK_HOST	“ROOT_OMTX”
WORD_01	CMND_LINK_CODE	
WORD_02	HUBS_LINK_PORT	32’ H0000000F
WORD_03	EDGE_LINK_PORT	32’ H000000FF
WORD_04	HUBS_LINK_DEST	HUBS_DEST_SCUT
WORD_05	EDGE_LINK_DEST	EDGE_DEST_SCUT
WORD_06	CORE_LINK_DEST	CORE_DEST_DVPX
WORD_07	CMND_LINK_AUX I	NA
WORD_08	MAST_META_DDRS	ROOT_META_DDRS
WORD_09	MAST_META_ROWS	ROOT_META_ROWS
WORD_10	MAST_META_COLS	ROOT_META_COLS
WORD_11	MAST_CMND_AUX I	NA
WORD_12	SLAV_META_DDRS	CORE_META_DDRS
WORD_13	SLAV_META_ROWS	CORE_META_ROWS
WORD_14	SLAV_META_COLS	CORE_META_COLS
WORD_15	SLAV_CMND_AUX I	NA

3.7 收集 CORE 矩阵数据到 HUB

报文 字段	缩写	CMD_1	CMD_2
WORD_00	CMND_LINK_HOST	“HUBS_IMTX”	“HUBS_IMTX”
WORD_01	CMND_LINK_CODE		
WORD_02	HUBS_LINK_PORT	32’ H0000000F	32’ H0000000F
WORD_03	EDGE_LINK_PORT	32’ H000000FF	32’ H000000FF
WORD_04	HUBS_LINK_DEST	HUBS_DEST_DDRS	HUBS_DEST_DDRS
WORD_05	EDGE_LINK_DEST	EDGE_DEST_SCUT	EDGE_DEST_SCUT
WORD_06	CORE_LINK_DEST	CORE_DEST_GSM0	CORE_DEST_GSM1
WORD_07	CMND_LINK_AUX I	ROOT_META_COLS	ROOT_META_COLS
WORD_08	MAST_META_DDRS	HUBS_META_DDRS	HUBS_META_DDRS
WORD_09	MAST_META_ROWS	HUBS_META_ROWS	HUBS_META_ROWS
WORD_10	MAST_META_COLS	HUBS_META_COLS	HUBS_META_COLS
WORD_11	MAST_CMND_AUX I	NA	NA
WORD_12	SLAV_META_DDRS	CORE_META_DDRS	CORE_META_DDRS
WORD_13	SLAV_META_ROWS	CORE_META_ROWS	CORE_META_ROWS
WORD_14	SLAV_META_COLS	CORE_META_COLS	CORE_META_COLS
WORD_15	SLAV_CMND_AUX I	NA	NA

3.8 收集 HUBS 矩阵数据到 ROOT

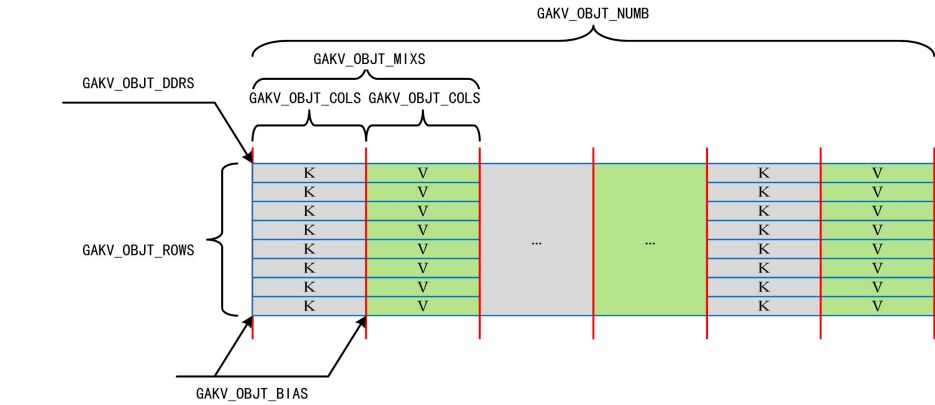
报文 字段	缩写	CMD_1
WORD_00	CMND_LINK_HOST	“ROOT_IMTX”
WORD_01	CMND_LINK_CODE	
WORD_02	HUBS_LINK_PORT	32’ H0000000F
WORD_03	EDGE_LINK_PORT	32’ H00000000
WORD_04	HUBS_LINK_DEST	HUBS_DEST_DDRS
WORD_05	EDGE_LINK_DEST	EDGE_DEST_SCUT
WORD_06	CORE_LINK_DEST	CORE_DEST_SCUT
WORD_07	CMND_LINK_AUX I	
WORD_08	MAST_META_DDRS	ROOT_META_DDRS
WORD_09	MAST_META_ROWS	ROOT_META_ROWS
WORD_10	MAST_META_COLS	ROOT_META_COLS
WORD_11	MAST_CMND_AUX I	NA
WORD_12	SLAV_META_DDRS	HUBS_META_DDRS
WORD_13	SLAV_META_ROWS	HUBS_META_ROWS
WORD_14	SLAV_META_COLS	HUBS_META_COLS
WORD_15	SLAV_CMND_AUX I	HUBS_META_DMMY

3.9 GAKV 广播下发矩阵数据

用于 GQA 模型的 KV 计算后数据的 degather. 自动进行 128 列的块数据交织

报文 字段	缩写	CMD_1
WORD_00	CMND_LINK_HOST	“HUBS_OMTX”
WORD_01	CMND_LINK_CODE	
WORD_02	HUBS_LINK_PORT	32’ H0000000F
WORD_03	EDGE_LINK_PORT	32’ H00000000
WORD_04	HUBS_LINK_DEST	HUBS_DEST_DDRS
WORD_05	EDGE_LINK_DEST	EDGE_DEST_SCUT
WORD_06	CORE_LINK_DEST	CORE_DEST_DVPX

WORD_07	CMND_LINK_AUX I	GAKV_OBJT_NUMB
WORD_08	MAST_META_DDRS	GAKV_OBJT_DDRS
WORD_09	MAST_META_ROWS	GAKV_OBJT_ROWS
WORD_10	MAST_META_COLS	GAKV_OBJT_COLS
WORD_11	MAST_CMND_AUX I	GAKV_OBJT_BIAS
WORD_12	SLAV_META_DDRS	CORE_META_DDRS
WORD_13	SLAV_META_ROWS	CORE_META_ROWS
WORD_14	SLAV_META_COLS	CORE_META_COLS
WORD_15	SLAV_CMND_AUX I	GAKV_OBJT_MIXS



以 GQA 的 KV 嵌入式编码 512 列（QWen-30-A3B），分布到 16 个 CORE 计算，每个核的 KV 计算 32 列为例，在提示词长度为 17 时，该指令的参数为：

报文 字段	缩写	CMD_1	说明
WORD_00	CMND_LINK_HOST	“HUBS_OMTX”	
WORD_01	CMND_LINK_CODE		
WORD_02	HUBS_LINK_PORT	32’ H0000000F	
WORD_03	EDGE_LINK_PORT	32’ H00000000	
WORD_04	HUBS_LINK_DEST	HUBS_DEST_DDRS	
WORD_05	EDGE_LINK_DEST	EDGE_DEST_SCUT	
WORD_06	CORE_LINK_DEST	CORE_DEST_DVPX	
WORD_07	CMND_LINK_AUX I	GAKV_OBJT_NUMB	16
WORD_08	MAST_META_DDRS	GAKV_OBJT_DDRS	
WORD_09	MAST_META_ROWS	GAKV_OBJT_ROWS	17
WORD_10	MAST_META_COLS	GAKV_OBJT_COLS	32
WORD_11	MAST_CMND_AUX I	GAKV_OBJT_BIAS	0/32
WORD_12	SLAV_META_DDRS	CORE_META_DDRS	
WORD_13	SLAV_META_ROWS	CORE_META_ROWS	136=17*4*2
WORD_14	SLAV_META_COLS	CORE_META_COLS	永远是 128
WORD_15	SLAV_CMND_AUX I	GAKV_OBJT_MIXS	64=32+32